YOUSEF R. M. ABUSULTAN 71605

GitHub Link: https://github.com/Yousef71605/Fall26AoA2026.git

4). Oct. 2026

Date:

Q1: Given an array of n integers, design an algorithm to find the largest and smallest elments in the array

Algorithm:

1. Algorithmic Strategy

We scan the array from the first elment to the last elment
We intially consider the first elment as both the smallest and largest elment.
Then, for each remaining elment, we compare it with the current smallest and largest values.
If the elment is smaller, we update smallest
If the elment is larger, we update largest

2. Algorithm + Pseudocode

Algorithm MINMAX(A, n, smallest, largest)
{ This algorithm is used to find the smallest and largest elements in array of n integers }
Step 1 [Initalize smallest and largest]
smallest = A[0]
largest = A[0]
Step 2 [Scan the remaining elments]
for(i=1; i<n; i++){
if (A[i] < smallest) then smallest = A[i] end if
if (A[i] > largest) then largest = A[i] end if
}
Step 3 [Display the Results
Write (smallest, largest)
Step 4 [finish]
Exit

Scanned with CamScanner

Date:

(2). Pseudocode C++

```
MINMAX(A,n,smallest,largest){
//Find the smallest and largest elment
smallest = A[0];
largest = A[0];
for(int i=0; i<n; i++){
 if(A[i]<smallest)
  smallest =A[i];
 if(A[i] > largest)
  largest = A[i];
   }
cout<<"Smallest=" << smallest <<endl;
cout<<"Largest =" <<largest <<endl;
}
```

(3). Major Primitive Operations

The major primitive operations:

1- Array access: A[i]
2- Comparison: A[i] < smallest ; $i<n$
3- Comparison: A[i] > largest
4- Assignment: smallest = A[0]; largest = A[0]
5) Arithmetic operation: Increment i++

Date:

(4.) Best Case and Worst-Case Running Time

Best case: Even in the best case, the algorithm must examine all n elements. For each element after the first element, two comparisons are performed.

Number of comprisons: $2(n-1)$ $2n-2$ $= n$ (C)

Therefor: Best Case = $\Omega(n)$

Worst Case: In the worst case, the algorithm also examines all n elements

Therefor Worst case = $O(n)$

* The best case and worst case are the same because the algorithm must examine every element to guarantee that the smallest and largest elements have been found.

(5) Asymptotic Notation

The running time can be expressed as:

$T(n) = 2(n-1) + C$

$= 2n-2+C$ by ignoring Constants and lower order

$T(n) = n$

Therefor:

Best Case = $\Omega(n)$
Average Case = $\Theta(n)$
Worst Case = $O(n)$

Scanned with CamScanner

YOUSEF K.M.AQUSULTAN 71605

Date:

6. Space Complexity

The algorithm uses only a constant number of additional variables:
smallest
largest
i

It doesn't create another array or additional data structure.
Therefor:

Space Complexity = 1 or C

YOUSEF K.M.ABUSULTAN 7/605

Given Problem:

Q2: Given n integers, design an algorithm to determine whether a particular value X exists in the array.

We will use two different strategies:

1. linear Search
2. Binary Search (requires sorted array)

A) Design of the First Algorithm:

Linear Search checks the elements one by one from the start of the array.

If an element is equal to X, the algorim returns that X exist.

If all the elements are checked and x is not found, the algorithm returns that x does not exist.

Algorithm:

Algorithm LINEARSEARCH(A, n, X, found)

{ The algorithm is used to determine weather the value X exists in an array }.

Step 1 [Initialize the search]

found = false;

Step 2 [Search for X]

for (i=0; i<n; i++) {

if (A[i] == X) then

found = true

break } }

Step 3 [Display the Result]

if (found == true) then Write ("X exists in the array")

else Write ("X doesn't exist in the array")

end if

Step 4 [finish]

Exit

YOUSEF K.M.ABUSULTAN 21605

7. Date:

Pseudocode C++

LINEARSEARCH(A,n,X){
// Search for X in the array
bool found = false;
for (int i=0; i<n; i++){
if(A[i] == X){
found = true;
break; } }
if(found == true)
cout << "X exists in the array";
else cout << "X doesn't exist in the array";
}

7. Design of the Second Algorithm

Binary Search checks the middle element of the array
If the middle element is equal to X, the value has been found
If X is smaller than the middle element, the search continues in the left half.
If X is larger than the middle element, the search continues in the right half.
The array must be sorted before applying Binary Search.

* Algorithm:

Algorithm BINARYSEARCH(A,n,X,found)
{This algorithm is used to determine whether the value X exists in a sorted array}
Step-1 [Initalize the search boundaries]
low=0
high=n-1
found=false

7. Date:

Step-2 [Search for X]
while(low<=high){
mid=(low+high)/2
if(A[mid]==X) then{
found=true
break }
else if(X < A[mid]) then{
high=mid-1
else low=mid+1
end if }
Step-3 [Display the Result]
if(found == true) then
Write("X exists in the array")
else
Write("X doesn't exist in the array")
Step-4 [Finish]
Exit

* Pseudocode:

BINARYSEARCH(A, n, X){
// search for X in a sorted array
int low=0;
int high=n-1; bool found=false;
While(low<=high)
int mid=(low+high)/2;
if(A[mid]==X){ found=true; break;}
else if(X<A[mid]) high=mid-1;}
else{ low=mid+1;
}}
if(found==true)
cout<<"X exists in the array";
else cout<<"X doesn't exist in the array";
}

Scanned with CamScanner

YOUSEF K.M.ABUSULTAN 71605

Date:

(8.) Best-Case and Worst-Case Time Complexity

Linear Search

Best Case:

The best case occurs when X is the first element of the array

Best Case = $\Omega(1)$

Worst Case:

The Worst case occurs when X is the last element or when X doesn't exist in the array.

Worst case = $O(n)$

Binary Search:

Best Case: The best case occurs when X is the middle element in the first comparison.

Best Case: $\Omega(1)$

Worst Case: In the worst case, the search space is divided approximately every step.

Worst Case = $O(\log n)$

(9.) Linear Search: Best Case = $\Omega(1)$

Worst Case = $O(n)$

~~Overall~~ Average = $\Theta(n)$

Binary Search: Best Case = $\Omega(1)$

Worst Case = $O(\log n)$

Average = $\Theta(\log n)$

(10) Complexity Comparison

Algorithm	Best Case	Worst Case	Average
Linear Search	$\Omega(1)$	$O(n)$	$\Theta(n)$
Binary Search	$\Omega(1)$	$O(\log n)$	$\Theta(\log n)$

Scanned with CamScanner

(11.) Comparison and Efficiency

* Linear Search:

Linear Search is simple and does not require the array to be sorted.

It is suitable for:

- Small arrays
- Unsorted arrays
- Situations where sorting is not necessary

Worst complexity is $O(n)$

* Binary Search:

Binary search is more efficient for large sorted arrays.

It reduces the search area by approximately half after each comparison.

Worst complexity is $O(\log n)$

Therefor, Binary Search is more efficient than Linear Search for large sorted arrays

KAGHAZ
www.kaghaz.pk
Scanned with CamScanner

YOUSEF K.M.ABUSULTAN 7/6/25
day / date:

(12.) Effect of Input Size and Arrangement

*Small Input: linear search can be suitable because the number of elements is small.

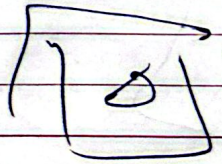
*Large Unsorted Input: linear Search is suitable because it can work directly.

*Large sorted Input: Binary Search is more efficient because it eliminates approximately half of the remaining elments at each step.
Binary searg = O (log n)

* Position of X
For Linear Search:
. If x is at the beginning → Sb(1)
. if x is at the end → O(n)
. if x does not exist → O(n)

For Binary Search:
. if x is the middle element intially → Sb (1)

In the worst case → O log (n)



KAGHAZ
www.kaghaz.pk

Scanned with CamScanner